

LEARNING RESOURCE GUIDE

ISC2 CCSP: Certified Cloud Security Professional (Aug 2026) | Professional Participant Workbook

Learning Objectives

By the end of this lecture and its accompanying guide, you will be able to:

- By the end of this lecture, you will be able to define threat modelling and explain why it is most effective during the design phase of the SDLC.
- By the end of this lecture, you will be able to apply the STRIDE framework to identify threats across all six threat categories for a given system component or data flow.
- By the end of this lecture, you will be able to use the DREAD framework to score and prioritise identified threats, enabling informed decisions about remediation sequencing.
- By the end of this lecture, you will be able to explain the complementary relationship between STRIDE and DREAD, and why using DREAD without prior threat identification produces incomplete results.
- By the end of this lecture, you will be able to conduct a basic threat modelling exercise using data flow diagrams and trust boundary analysis for a cloud application.

Introduction

Threat modelling is the practice of reading an attacker's thinking before they ever touch your system. It is a structured discipline that allows security professionals to identify potential threats to a system, understand how those threats could be exploited, and design controls to prevent or mitigate them — all before the system exists in its final form. The economic argument for threat modelling is compelling: a design flaw identified in a whiteboard session costs hours to address. The same flaw identified during a penetration test after deployment costs significantly more, and the same flaw identified after a breach costs more still. Threat modelling is one of the most cost-effective security activities a development team can perform.

The CCSP exam tests two specific threat modelling frameworks in the Cloud Application Security domain: STRIDE and DREAD. STRIDE is a categorisation framework that helps security teams systematically think through the types of threats a system might face. DREAD is a risk scoring framework used to prioritise the threats that STRIDE — or any other identification method — has surfaced. They are designed to work together: STRIDE finds the threats, DREAD ranks them. Understanding both frameworks, and the relationship between them, is essential for applying structured threat modelling to cloud application development.

For Cloud Security Practitioners, the most important practical skill is knowing when and how to conduct a threat modelling session. Threat modelling is most effective during the design phase of the SDLC, when the system's data flow diagram has been sketched, its components and integrations are defined, and architectural decisions are still flexible. It does not require a complete, implemented system — only a clear picture of what the system will do, what data it will handle, who will access it, and how its components will communicate. The outputs of a threat modelling session — a prioritised list of threats and the design decisions made to address them — become part of the design artefact and inform every subsequent phase of development.

Core Concepts Explained

Threat Modelling: Definition and Optimal Timing

Definition: Threat modelling is the structured practice of identifying potential threats to a system, understanding how those threats could be exploited, and designing controls to mitigate them before the system is built. It is most effective during the design phase of the SDLC, when architecture decisions are still cheap to change.

Business relevance: A design flaw caught during threat modelling costs hours to fix. The same flaw caught during penetration testing costs days and potential architectural rework. The same flaw caught after a breach costs orders of magnitude more, including regulatory penalties, remediation costs, and reputational damage.

Cloud Security Practitioner relevance: Cloud Security Practitioners lead or facilitate threat modelling sessions during design reviews, prepare data flow diagrams identifying trust boundaries as the session input, and ensure that the session's outputs — prioritised threats and design decisions — are documented and reflected in the design specification.

Real-world example: Google's secure development practices for its cloud services embed threat modelling as a standard step in design reviews rather than an optional exercise reserved for high-risk projects. When a Google Cloud team designs a new service feature that introduces a new data flow or trust boundary, the design review includes a structured threat modelling session. Security engineers review the proposed data flow diagram and apply threat identification frameworks to identify architectural risks before any implementation work begins. Google's published security research has documented that this practice — systematically reviewing data flows and trust boundaries in system designs before implementation — enables identification of architectural risks that would be far more costly to address after deployment. The investment is measured in engineer-hours during design; the avoided cost is measured in weeks of remediation or, in the worst case, post-breach response.

Risks or limitations: The risk of skipping design-phase threat modelling is that security decisions are made implicitly during implementation — by individual developers making local choices without a global view of the threat landscape. Implicit security decisions are inconsistent, undocumented, and frequently wrong. The best practice is making threat modelling a mandatory gate in the design phase exit, requiring a documented threat model before implementation begins.

STRIDE: A Systematic Threat Categorisation Framework

Definition: STRIDE is a threat categorisation framework where each letter represents a threat category: Spoofing (impersonating a user or service), Tampering (modifying data or code without authorisation), Repudiation (denying actions due to absence of audit trail), Information Disclosure (exposing sensitive data to unauthorised parties), Denial of Service (disrupting service availability), and Elevation of Privilege (gaining permissions beyond what was granted).

Business relevance: STRIDE's value is its completeness. Teams brainstorming threats in an unstructured way tend to focus on the threats they already fear most. STRIDE ensures that all six threat categories are reviewed systematically, preventing the failure mode of missing entire categories of risk.

Cloud Security Practitioner relevance: Cloud Security Practitioners use STRIDE to structure threat modelling sessions, walking through each system component and data flow and asking the six STRIDE questions at each trust boundary. The output is a comprehensive list of potential threats that can then be evaluated for likelihood and impact.

Real-world example: A cloud security team conducting a STRIDE analysis of a new patient data API built on AWS API Gateway and Lambda worked through the six categories systematically for the API endpoint that received patient record update requests. For Spoofing: could an attacker impersonate an authorised clinical user to submit fraudulent record updates? The existing control — JWT authentication — was assessed, and the team identified that the token validation logic did not check token expiry, allowing a stolen token to be used indefinitely. For Tampering: could an attacker modify a record update request in transit? TLS was in place, but the team identified that the API did not validate that the patient ID in the request body matched the patient ID in the JWT claim, allowing an authenticated user to modify another patient's records. For Repudiation: did the API generate an audit log entry for every record modification? Review revealed that Lambda function invocations were logged in CloudWatch but that the logs did not capture the specific fields modified — insufficient for HIPAA audit trail requirements. For Information Disclosure: could the API error responses reveal patient data or internal system details? Review found that Lambda error responses included full stack traces. For Denial of Service: was the API throttled? No throttling was configured on API Gateway. For Elevation of Privilege: could a standard user trigger admin-level operations? Review found that admin operations were filtered by a role check in the Lambda function that could be bypassed by directly invoking the Lambda ARN. Six design vulnerabilities — one per STRIDE category — identified in a two-hour session.

Risks or limitations: The risk of unstructured threat identification is threat blindness — teams consistently miss entire threat categories, most commonly Repudiation and Elevation of Privilege. The best practice is using a formal framework like STRIDE as a checklist, working through every category for every trust boundary before concluding the session.

STRIDE Trust Boundary Analysis

Definition: Trust boundaries are the points in a system where data crosses from one security domain to another — from an external user to the application, from the application to a database, from one service to another. STRIDE is applied at each trust boundary to identify what threats exist at that crossing point.

Business relevance: Trust boundaries are the highest-concentration points of attack surface in any system. An attacker who can manipulate what crosses a trust boundary — by spoofing the identity of the caller, tampering with the data in transit, or escalating privileges through the crossing — has the highest leverage on the system's security.

Cloud Security Practitioner relevance: Cloud Security Practitioners prepare or review data flow diagrams of the system under analysis, identify every trust boundary — every point where data crosses between security domains — and ensure that STRIDE is applied at each boundary before the modelling session concludes.

Real-world example: A cloud application team building an API-driven e-commerce platform on Azure identified four trust boundaries in their data flow diagram: the external customer browser to the Azure API Management gateway (public internet to managed API layer), the API gateway to the application's Azure App Service backend (managed API to application compute), the backend to the Azure SQL Database (application compute to data store), and the backend to a third-party payment processing API (internal application to external payment service). Applying STRIDE at the payment processing trust boundary revealed that the integration used a shared API key stored as a plaintext environment variable in the App Service configuration — a Spoofing threat (an attacker with App Service console access could read the key and impersonate the application's payment requests) and an Information Disclosure threat (the key was visible in App Service logs). The team moved the API key to Azure Key Vault with managed identity access before the sprint ended, eliminating both threats before they reached production.

Risks or limitations: The risk of incomplete trust boundary identification is that threat modelling misses the crossing points where the most significant risks are concentrated. The best practice is generating the data flow

diagram before the threat modelling session, having a second reviewer verify that all trust boundaries are marked, and not beginning the STRIDE analysis until boundary completeness is confirmed.

DREAD: Risk Scoring and Threat Prioritisation

Definition: DREAD is a risk scoring framework that assigns a numerical score to identified threats across five dimensions: Damage Potential (how severe is the impact if exploited?), Reproducibility (how easily can the exploit be repeated?), Exploitability (how much skill or effort is required?), Affected Users (how many users or systems are impacted?), and Discoverability (how likely is an attacker to find this vulnerability?). Each dimension is scored 1-10; the sum is the DREAD score.

Business relevance: DREAD converts a list of identified threats into a prioritised remediation backlog. Without prioritisation, development teams treat all threats as equally urgent — which means limited security effort is allocated inefficiently. DREAD enables the team to address the highest-risk threats first.

Cloud Security Practitioner relevance: Cloud Security Practitioners apply DREAD scores to the threat list produced by STRIDE, facilitate the scoring discussion with the development team, and use the resulting prioritised list to define the remediation backlog — including which threats are accepted, which are mitigated by design, and which are transferred to compensating controls.

Real-world example: Following a STRIDE analysis of a cloud-based identity management application on GCP, the security team had a list of 14 identified threats. Applying DREAD to prioritise them produced a clear remediation sequence. The highest-scoring threat (DREAD score 44/50) was an Elevation of Privilege finding: an unauthenticated user could access the administrative API by guessing sequential object IDs, bypassing role-based access controls. The dimensions were scored as: Damage Potential 10 (full admin access), Reproducibility 10 (trivially repeatable), Exploitability 9 (requires only a browser), Affected Users 8 (all user accounts exposed), Discoverability 7 (the pattern is well-known to attackers). This threat was assigned to the next sprint for immediate remediation. A lower-scoring threat (DREAD score 18/50) — a timing side-channel in the password comparison function — was assessed as Exploitability 3 (requires sophisticated tooling), Reproducibility 4 (requires specific timing conditions), and was scheduled for the following sprint. DREAD converted 14 undifferentiated threats into a sequenced remediation plan that directed effort to the vulnerabilities representing the highest real-world risk.

Risks or limitations: The risk of misapplying DREAD is scoring it inconsistently across the team — one person's 'exploitability 3' is another's 'exploitability 7.' The best practice is establishing scoring rubrics for each dimension before the session (for example: Exploitability 9-10 = exploitable with a browser and no credentials; Exploitability 4-6 = requires custom tooling; Exploitability 1-3 = requires nation-state capability) and scoring as a group to normalise interpretations.

STRIDE and DREAD as Complementary Frameworks

Definition: STRIDE and DREAD are complementary, not competing. STRIDE is a threat identification framework: it systematically surfaces threats. DREAD is a threat prioritisation framework: it ranks identified threats by risk score. DREAD only works after a threat list exists — using DREAD without a prior identification step produces a well-ordered list of only the threats the team already knew about.

Business relevance: Teams that attempt to prioritise threats without first identifying them systematically — jumping to scoring before completing identification — consistently fail to surface the threats they had not already anticipated. The identification step (STRIDE) is not separable from the prioritisation step (DREAD) if the goal is a complete threat model.

Cloud Security Practitioner relevance: Cloud Security Practitioners structure threat modelling engagements in two phases: first the STRIDE identification phase (which produces a complete threat list), then the DREAD

prioritisation phase (which produces a sequenced remediation backlog). They resist pressure to skip or abbreviate the identification phase in the interest of moving quickly to prioritisation.

Real-world example: A development team under time pressure to complete their design phase asked their security consultant to skip the STRIDE analysis and proceed directly to scoring the three threats they had already identified. The consultant conducted the STRIDE analysis anyway — it took 90 minutes. It surfaced 11 additional threats, including two in the Repudiation category (the team's application had no audit logging for data modification events, a HIPAA requirement they had overlooked) and three in the Elevation of Privilege category (the application's service account had been granted owner-level permissions to the GCP project rather than the minimum required for its function). Had the team proceeded directly to DREAD scoring on their original three threats, both the HIPAA logging gap and the over-privileged service account would have reached production undetected. The 90-minute STRIDE session avoided what the consultant estimated would have been weeks of remediation and a potential HIPAA breach notification incident.

Risks or limitations: The risk of skipping the identification phase and going directly to prioritisation is a false sense of completeness — the team believes they have modelled their threats when they have only scored the threats they already feared. The best practice is always completing the full STRIDE pass before beginning DREAD scoring, and treating the STRIDE output as the input to DREAD, not as optional context.

Practical Applications

Threat modelling using STRIDE and DREAD is a practical skill that Cloud Security Practitioners apply in design reviews, sprint planning, and architecture assessments for cloud applications.

Facilitating a STRIDE Session for a Cloud API Design

A Cloud Security Practitioner is asked to conduct a threat modelling session for a new REST API that will expose patient appointment scheduling data to third-party healthcare applications. Before the session, the practitioner prepares a data flow diagram showing four components: the mobile and web clients, the API gateway, the appointment scheduling service running in a container on Kubernetes, and the PostgreSQL database storing appointment records. They mark three trust boundaries: external internet to API gateway, API gateway to scheduling service, and scheduling service to database. In the session, they work through STRIDE at each trust boundary. At the external internet boundary, Spoofing surfaces a finding: the API uses API keys for third-party authentication, but there is no IP allowlisting or mTLS — a compromised API key gives unrestricted access from any source. Tampering surfaces that the API does not validate the Content-Type header, enabling content-sniffing attacks. Repudiation surfaces that there is no audit log of which third-party application accessed which patient's records. Information Disclosure surfaces that error responses include internal service names. Denial of Service surfaces that API Gateway has no rate limiting configured. Elevation of Privilege surfaces that a third-party with access to one patient's records could access any patient's records by incrementing the patient ID in the request path. Six threats in two hours across three trust boundaries. Each becomes a design decision to address before the sprint that implements the API begins.

Applying DREAD to Prioritise a Threat Remediation Backlog

Following a STRIDE analysis of a cloud-based document management platform, the security team has identified 16 threats. The Cloud Security Practitioner facilitates a DREAD scoring session to convert the list into a prioritised remediation backlog. They establish scoring rubrics first: Damage Potential 10 = full data breach of all documents, 5 = breach of a subset, 1 = no sensitive data affected. Reproducibility 10 = exploitable reliably every time, 5 = exploitable under specific conditions, 1 = requires exact timing. The team scores each threat across all

five dimensions. The top-ranked threat (score 46/50) is an authentication bypass that allows an unauthenticated user to access documents by manipulating the session token — high damage, fully reproducible, exploitable with a free tool, affects all users, publicly known technique. This is assigned as a P0 in the next sprint with a 24-hour resolution SLA. The lowest-ranked threat (score 12/50) is a weak randomness source in a non-security-critical ID generation function — low damage, not reproducible without source code access, requires significant expertise, affects no users directly. This is scheduled for the following quarter. The DREAD scores convert 16 threats of varying severity into an ordered remediation plan that the development team can slot into sprints based on priority without additional security judgement at the point of scheduling.

Integrating Threat Modelling into the Agile Sprint Cycle

A Cloud Security Practitioner working with an Agile cloud development team establishes a lightweight threat modelling process that runs within the sprint planning ceremony rather than as a separate engagement. For each sprint that introduces new functionality — a new API endpoint, a new cloud service integration, a new data flow — the practitioner allocates 30 minutes in sprint planning to review the relevant data flow diagram and apply the STRIDE checklist to any new trust boundaries introduced by the sprint's work. They maintain a running threat register in the team's Jira project as a dedicated security epic, where new threats from each sprint's analysis are added as issues with DREAD scores and priority labels. Threats assessed as critical are added to the active sprint immediately as blocking stories. Threats assessed as high are added to the next sprint's backlog. Medium and low threats accumulate in the security epic for periodic review. Over six months, this process produces a continuously updated threat model that reflects the evolving architecture of the application, rather than a single threat modelling document that becomes stale after the first post-launch sprint. The team's security posture improves continuously rather than being assessed only at major project milestones.

Best Practices

- **Conduct threat modelling during the design phase, before implementation begins.** The design phase is when threat modelling delivers the highest return on investment — architectural decisions are still flexible, and the cost of making changes is measured in hours rather than weeks of rework. Make threat modelling a required gate in the design phase exit: no implementation begins until a documented threat model has been produced and reviewed.
- **Always complete STRIDE identification before beginning DREAD prioritisation.** Teams under time pressure frequently skip the identification phase and jump to scoring threats they already know about. This produces a well-ordered list of known threats while leaving unknown threats — often the most dangerous — undetected. The STRIDE identification phase is not separable from the prioritisation phase. Treat it as a prerequisite, not optional context.
- **Work from a data flow diagram with all trust boundaries marked before the session begins.** Trust boundaries are the primary analytical unit in threat modelling — they are where threats are concentrated and where controls must be placed. Attempting to identify threats without a data flow diagram produces inconsistent and incomplete results. Prepare the diagram before the session and have a second reviewer confirm that all trust boundaries are marked.
- **Establish DREAD scoring rubrics before the session, and score as a group.** DREAD scores are meaningless without consistent interpretation of the scoring dimensions. Define what constitutes a 9 vs. a 5 for each dimension before the scoring session begins. Score each threat as a group discussion rather than allowing individual scores to diverge without discussion. Inconsistent scoring produces a prioritised list that reflects individual interpretations rather than shared risk judgement.

- **Document the threat model as a formal artefact and maintain it as the application evolves.** A threat model produced during initial design becomes stale after the first significant sprint adds new functionality, integrations, or data flows. Maintain the threat model as a living document — updating it whenever the architecture changes significantly — and treat the threat register as a backlog that is reviewed and groomed alongside the product backlog.

Common Mistakes and Misconceptions

Misconception: Treating threat modelling as an optional activity reserved for high-risk projects or final-stage review.

Correct approach: Threat modelling is most valuable precisely on projects where it seems least urgent — where the team is confident in their design and has not explicitly considered what could go wrong. The best practice is making threat modelling a mandatory design phase gate for every project that introduces new data flows, new trust boundaries, or new cloud service integrations.

Misconception: Using DREAD to prioritise a small list of pre-known threats without first conducting a systematic STRIDE identification.

Correct approach: DREAD is a prioritisation tool, not a discovery tool. It can only rank threats that have already been identified. Teams that jump to DREAD scoring without first completing a STRIDE analysis produce an efficiently prioritised list of the threats they already knew about — while leaving everything else unaddressed. Always run STRIDE first.

Misconception: Conducting a STRIDE analysis without a data flow diagram or without explicitly identifying trust boundaries.

Correct approach: STRIDE applied to a vague description of a system produces vague, incomplete results. The best practice is preparing a data flow diagram before the session, marking every trust boundary — every point where data crosses between security domains — and applying STRIDE systematically at each boundary rather than to the system as a whole.

Misconception: Producing a threat model once during initial design and never updating it as the application evolves.

Correct approach: An outdated threat model is worse than no threat model — it creates a false sense of security coverage while the actual threat surface has expanded beyond what the model addresses. The best practice is treating the threat model as a living document, updated whenever significant architecture changes introduce new components, data flows, or trust boundaries.

table: STRIDE Threat Category Reference with Cloud Examples

Use this table as a quick reference during threat modelling sessions. For each trust boundary in the system's data flow diagram, apply the six STRIDE questions and note any findings in the threat register.

STRIDE Category	Threat Description	Cloud Application Example	Typical Controls
Spoofing	Attacker impersonates a legitimate user, service, or system to gain unauthorised access	Attacker uses a stolen OAuth token or API key to impersonate an authorised service calling a cloud API	Strong authentication (mTLS, OAuth 2.0 with short-lived tokens); IP allowlisting; token binding

Tampering	Attacker modifies data, configuration, or code without authorisation	Attacker modifies request body in transit to change transaction amount or patient record fields	TLS for all data in transit; input validation and integrity checks; digital signatures on sensitive payloads
Repudiation	User or system denies having performed an action due to insufficient audit trail	A privileged user deletes records and denies it because CloudTrail logging was not enabled for the S3 bucket	Comprehensive audit logging for all sensitive actions; immutable log storage; log integrity controls
Information Disclosure	Sensitive data is exposed to unauthorised parties	Error responses include stack traces with database connection strings; S3 bucket set to public access	Generic error responses; encryption at rest with CMK; least privilege access policies; S3 Block Public Access
Denial of Service	Attacker disrupts availability for legitimate users	Unauthenticated API endpoint receives flood of requests; Lambda cold-start behaviour exploited to exhaust concurrency	API rate limiting; WAF rules; auto-scaling; request authentication before resource allocation
Elevation of Privilege	Attacker gains permissions beyond what was granted	Service account with owner-level IAM permissions; IDOR vulnerability allows user to access another user's records	Least privilege IAM roles; object-level authorisation checks; privilege boundary controls

How to use this: Apply all six categories to every trust boundary in the data flow diagram. A category with no findings should still be documented as 'reviewed — no finding' to provide evidence that the analysis was complete. Missing a category — especially Repudiation and Elevation of Privilege, which teams most frequently overlook — is the primary failure mode of unstructured threat identification.

Knowledge Check

Test your understanding before moving on. Answers are shown below each question.

Q1. A development team is designing a cloud application and wants to conduct threat modelling. The project manager suggests waiting until the testing phase, after the application is built, to conduct the threat model. Which of the following best describes the problem with this approach?

- A) A. Threat modelling requires a running application to be effective, so waiting until after testing is actually the correct approach.
- B) B. Threat modelling is most valuable during the design phase, when architectural decisions are still flexible and changes are cheap; waiting until after implementation means architectural flaws can only be addressed through expensive rework.
- C) C. Threat modelling should be conducted during the requirements phase, not the design phase, so waiting until testing is two phases too late.
- D) D. Threat modelling cannot be conducted until a penetration test has been completed to provide the initial finding list.

Answer: B. Threat modelling is most valuable during the design phase, when architectural decisions are still flexible and changes are cheap; waiting until after implementation means architectural flaws can only be addressed through expensive rework.

Q2. A security team conducting a STRIDE analysis of a cloud application's administrative API identifies a finding in the Repudiation category. Which of the following findings correctly corresponds to the Repudiation threat category?

- A) A. An attacker can exploit an IDOR vulnerability to access other users' administrative records.
- B) B. The administrative API does not enforce rate limiting, allowing an attacker to make unlimited requests.
- C) C. The administrative API does not generate audit log entries for configuration changes, meaning an administrator can modify settings and deny having done so.
- D) D. API error responses include internal service names, revealing the application's architecture to an attacker.

Answer: C. The administrative API does not generate audit log entries for configuration changes, meaning an administrator can modify settings and deny having done so.

Q3. A cloud application threat model has produced 12 identified threats. The security team wants to use DREAD to prioritise remediation. One threat — an unauthenticated path to administrative functions — scores 9 on Damage Potential, 10 on Reproducibility, 10 on Exploitability, 9 on Affected Users, and 8 on Discoverability. A second threat — a timing side-channel in a low-sensitivity function — scores 2, 3, 3, 1, and 4 respectively. What does DREAD indicate about remediation priority?

- A) A. Both threats should be remediated in the same sprint because both were identified in the same STRIDE session.
- B) B. The unauthenticated administrative path (total score 46) should be remediated with high urgency; the timing side-channel (total score 13) can be scheduled at a lower priority.
- C) C. The timing side-channel should be remediated first because low-scoring threats are often overlooked.
- D) D. DREAD scores only indicate technical severity, not business risk, so the product owner should determine remediation priority independently.

Answer: B. The unauthenticated administrative path (total score 46) should be remediated with high urgency; the timing side-channel (total score 13) can be scheduled at a lower priority.

Q4. Which of the following most accurately describes the relationship between STRIDE and DREAD?

- A) A. STRIDE and DREAD are alternative frameworks that perform the same function; organisations should choose one based on their development methodology.
- B) B. STRIDE identifies the types of threats a system may face; DREAD prioritises the identified threats by risk score. They are complementary: STRIDE produces the threat list that DREAD prioritises.
- C) C. DREAD is used during the requirements phase; STRIDE is used during the testing phase. Together they cover the full SDLC.
- D) D. STRIDE is applied to cloud applications; DREAD is applied to on-premises applications. They address different deployment environments.

Answer: B. STRIDE identifies the types of threats a system may face; DREAD prioritises the identified threats by risk score. They are complementary: STRIDE produces the threat list that DREAD prioritises.

Q5. A security team conducts a STRIDE analysis of a cloud application but does not prepare a data flow diagram before the session. They instead discuss the system verbally during the session. Which risk does this approach introduce?

- A) A. The session will produce too many threat findings to score with DREAD effectively.
- B) B. The team may miss trust boundaries that are not verbally described, leaving significant portions of the attack surface unanalysed.

- C) C. Without a data flow diagram, STRIDE can only be applied to the Spoofing and Tampering categories.
- D) D. Verbal descriptions are prohibited by CCSP threat modelling standards, which require formal UML diagrams.

Answer: B. The team may miss trust boundaries that are not verbally described, leaving significant portions of the attack surface unanalysed.

Reflection Questions

1. The lecture uses the pre-flight checklist analogy to explain STRIDE: a structured list that ensures the conversation covers all threat categories, in order, every time, regardless of experience or confidence. Where else in security practice do you see this principle — structure compensating for the unreliability of human memory and attention?
2. DREAD converts a list of threats into a prioritised remediation backlog. Think about a security finding you have encountered in your professional experience. How would you have scored it on the five DREAD dimensions? Does the resulting score match the priority the finding was actually given at the time?
3. The lecture describes a team that skipped the STRIDE identification phase and proceeded directly to DREAD scoring, discovering two significant threats that would have reached production undetected. What organisational pressures typically drive teams to skip or abbreviate identification steps? How would you push back?
4. Google's embedding of threat modelling into design reviews as a standard step, not a high-risk optional exercise, is described as a cultural norm rather than a procedural requirement. What is the difference between a cultural norm and a procedural requirement in a security context, and which produces better outcomes?
5. The lecture states that the goal of threat modelling is not perfect invulnerability but deliberate, informed design decisions with full awareness of residual risk. How does this framing change how you think about the outputs of a threat modelling session? What does a 'good' threat model look like if perfection is not the goal?

Key Takeaways

- Threat modelling is the practice of identifying potential threats, understanding how they could be exploited, and designing controls — before the system is built. Design-phase threat modelling is the most cost-effective security activity a development team can perform.
- STRIDE categorises threats into six categories: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege. Its value is completeness — applying all six categories prevents teams from missing entire threat classes.
- STRIDE is applied at trust boundaries — the points in a data flow diagram where data crosses between security domains. Preparing a data flow diagram with all trust boundaries marked before the session is a prerequisite for a complete STRIDE analysis.
- DREAD scores threats across five dimensions: Damage Potential, Reproducibility, Exploitability, Affected Users, and Discoverability. The sum provides a relative risk ranking that converts an unordered threat list into a prioritised remediation backlog.
- STRIDE and DREAD are complementary, not competing: STRIDE identifies threats, DREAD prioritises them. Using DREAD without a prior STRIDE analysis produces a well-prioritised list of only the threats the team already knew about.

- Threat models must be maintained as living documents — updated whenever architecture changes introduce new components, data flows, or trust boundaries. A stale threat model is worse than no threat model, because it creates a false sense of coverage.
- The goal of threat modelling is not perfect invulnerability but deliberate, informed design decisions with documented awareness of residual risk — which itself functions as a security control by preventing the most common failure mode: building without asking what could go wrong.

Recommended Next Actions

Put this lecture to work within the next seven days:

6. Select one cloud application you are responsible for or familiar with. Draw a basic data flow diagram showing its major components and mark every trust boundary. Use the STRIDE reference table in this guide to apply the six threat categories to at least one trust boundary and document your findings.
7. Review the threat findings you identified in the exercise above. Apply DREAD scoring to each finding using the dimension descriptions in this guide. Document your scores and the resulting priority order. Compare the priority order to how you would have intuitively ordered the findings without scoring.
8. Identify one upcoming design review on your team's calendar and propose a 45-minute STRIDE threat modelling session as part of the design review agenda. Prepare the data flow diagram for the session and share it with participants in advance so they can review it before the meeting.
9. If your team maintains a security backlog or risk register, audit it for completeness against the six STRIDE categories. Are there categories with no findings? That is more likely to indicate incomplete identification than a genuine absence of risk. Conduct a targeted STRIDE pass for any underrepresented categories.